Code review — rares-company-management

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect Spring Boot 4.1.1 / Java 21, temă JPA 1—* (`Department` 1—* `Employee`) expusă prin REST.

Runda 6 — 2026-10-20 — sha `86bdf3b`

Poarta de compilare: ✅ trece. **Verificat prin rulare:** pornit pe H2 cu `show-sql`, ca să număr interogările pe fiecare cerere, nu doar să citesc răspunsurile. Două commit-uri noi față de runda 5: `7dbcc59` (paginare + `GET /api/department/{id}/employees`) și `86bdf3b` .

Ce s-a întâmplat cu runda 5

Din	Constatare	Stare
R5 B1	N+1 scris de mână în `getAllEmployeeCounts`	✅ **ÎNCHIS** (`group by` : `GET /api/department/employee-count` emite 1 SQL, `Legal` = 0)
R5 B2	`@NamedEntityGraph` nefolosit, interogări în plus la `findById`	🔴 **DESCHIS**, vezi B2 mai jos
R5 B3	fără `@RestControllerAdvice`	🔴 **DESCHIS, a șasea rundă**, vezi B3
R5 B4	`Map<String,Object>` cu cheia `"emmployee count"`	🔴 **DESCHIS** (`{"emmployee count":3,"department":"IT"}` , 3 SQL)

R5 B1 e închis curat: `Legal` apare cu `employeeCount: 0` , deci `left join` + `count(e.id)` sunt corecte.

🔴 Critice

B1 — Paginarea nouă `/api/employees/pageable` are N+1 pe `department`

`employee/services/EmployeeQueryServiceImpl.java:61`

CUM E ACUM. `employeeRepository.findAll(pageable)` aduce angajații fără departament, iar `EmployeeResponse.from` (`EmployeeResponse.java:16`) cheamă `getDepartment().getName()` pentru fiecare. Pe o pagină cu 7 angajați:

```
GET /api/employees/pageable?size=7   ->  200, 5 SQL
   select ... from employee e1_0 order by e1_0.salary offset ? rows fetch first ? rows only
   select count(e1_0.id) from employee e1_0
   select d1_0.id,... from department d1_0 where d1_0.id=?     (x3)
```

Sunt 5 interogări în loc de 2. Este exact cazul din „Regulă de reținut" (jos): returnezi `Page` , deci `@EntityGraph` .

DE CE. `department` e `@ManyToOne(fetch = LAZY)` , așa că Hibernate pune în câmp un proxy, un obiect gol care știe doar id-ul. Primul `getName()` pe proxy declanșează un SELECT. De ce 3 și nu 7: contextul de persistență ține minte entitățile după id, deci al doilea angajat din IT primește proxy-ul deja încărcat. N-ul tău e numărul de departamente **distincte** de pe pagină. Cu 3 departamente în seeder nu se vede; cu 50

de departamente și `size=50` ajungi la 52 de interogări pe o singură cerere. Cu `@EntityGraph`, join-ul intră în SELECT-ul paginii, iar `count`-ul rămâne curat (7, nu rânduri de join).

FIX — în `EmployeeRepository`, graful pe toate metodele care ajung la `EmployeeResponse.from`:

```
@EntityGraph(attributePaths = "department")
Page<Employee> findAll(Pageable pageable);

@EntityGraph(attributePaths = "department")
List<Employee> findByJobTitle(String jobTitle);

@EntityGraph(attributePaths = "department")
Optional<Employee> findById(Long id);
```

Verificat pe o copie în scratchpad: `pageable?size=7` emite 2 SQL (`select ... from employee e1_0 left join department d1_0 ...` + `select count(e1_0.id) from employee e1_0`), iar `/api/employees/4` și `?jobTitle=Recruiter` emit câte 1.

+ același bug la:

- `EmployeeRepository.java:19` `findByJobTitle(String)` fără graf, consumat în `EmployeeQueryServiceImpl.java:53–55`: `GET /api/employees?jobTitle=Recruiter` = 2 SQL în loc de 1.
- `EmployeeQueryServiceImpl.java:28–37` `getEmployees()` cheamă `findAll()` **de două ori** (linia 31 doar pentru `isEmpty()`, linia 34 pentru date) plus N+1 la linia 36: `GET /api/employees` = 5 SQL (2 employee + 3 department). Fix: un singur apel, `List<Employee> l = ...; if (l.isEmpty()) throw ...`, cu `findAll()` redeclarat cu graf.
- `EmployeeQueryServiceImpl.java:47–51` ramura `jobTitle == null || isEmpty()`: `findAll()` + `EmployeeResponse::from` (linia 50). `GET /api/employees?jobTitle=` = 4 SQL.
- `EmployeeQueryServiceImpl.java:41–42` `getEmployee(Long id)`: `findById` fără graf. `GET /api/employees/4` = 2 SQL.
- `EmployeeCommandServiceImpl.java:123` + `:130` `deleteEmployee`: `findById` fără graf, apoi `employee.getDepartment().getName()`. Nu e N+1 (un singur rând), dar îl rezolvă același `findById` cu graf. Citit în cod, nereprodus.
- `DepartmentResponse.java:17–19` (folosit de `DepartmentQueryServiceImpl.java:40–41`): `findById` fără graf, apoi `getEmployees()` lazy: `GET /api/department/1` = 2 SQL (1+1, nu N+1; angajații au departamentul deja în sesiune). Vezi B2.
- `DepartmentQueryServiceImpl.java:66` + `:68`: `getDepartmentById(...)` chemat de două ori, două SQL identice. Vezi B2.
- Corecte, nu se ating: `DepartmentRepository.java:33` (`@EntityGraph` pe `getDepartmentById`), `:16–19` (`join fetch` pe `getDepartmentsWithEmployees`). Nicio apariție nouă în `DataSeeder.java` sau `DepartmentCommandServiceImpl.java`.

B2 — `@EntityGraph` pus pe o metodă nouă, nu pe `findById`, și metoda e chemată de două ori

`department/services/DepartmentQueryServiceImpl.java:66`

CUM E ACUM. Ai scris `@EntityGraph(attributePaths = "employees") Department getDepartmentById(Long id)` (`DepartmentRepository.java:32–33`), deci forma corectă cu `attributePaths`. Doar endpoint-ul nou o folosește, și o cheamă de două ori: o dată pentru `== null`, o dată ca să ia obiectul (`:66` și `:68`).

```
GET /api/department/2/employees   ->  200, 2 SQL, ambele identice:
   select d1_0.id,d1_0.budget,e1_0.department_id,... from department d1_0
     left join employee e1_0 on d1_0.id=e1_0.department_id where d1_0.id=?

GET /api/department/1             ->  200, 2 SQL (la fel ca în runda 5):
   select ... from department d1_0 where d1_0.id=?
   select ... from employee e1_0 where e1_0.department_id=?
```

`@NamedEntityGraph` a rămas nefolosit pe entitate (`Department.java:16–19`); `grep -rn EntityGraph src/` arată tot `DepartmentRepository.java:3`, `:32` și `Department.java:16`.

DE CE. Ai reparat locul nou și ai lăsat locul semnalat. Constatarea din runda 5 era despre `findById`, folosit de `GET /api/department/{id}`. Dublarea are și ea un mecanism: cache-ul de prim nivel scurtcircuitează doar `em.find(id)`. O metodă derivată `getDepartmentById` e o

interogare JPQL, iar o interogare merge mereu la bază, chiar dacă entitatea cerută e deja în memorie. De-aia vezi în consolă același SELECT de două ori. Tiparul `if (repo.get(x) == null) throw; obj = repo.get(x);` e cel pe care l-ai eliminat în runda 4 (M3, `existsById` + `findById`), revenit sub alt nume.

FIX — redeclari `findById` cu graf în `DepartmentRepository` și ștergi `getDepartmentById` și `@NamedEntityGraph`:

```
@EntityGraph(attributePaths = "employees")
Optional<Department> findById(Long id);
```

Apoi, în serviciu, un singur apel:

```
Department department = departmentRepository.findById(departmentId).orElseThrow(DepartmentNotFound::new);
```

Verificat în scratchpad: `/api/department/1` și `/api/department/1/employees` emit câte 1 SQL.

+ același bug la:

- `DepartmentQueryServiceImpl.java:41` (`getDepartmentById`): `findById` + `DepartmentResponse.from` iterează `getEmployees()` lazy = 2 SQL.
- `DepartmentQueryServiceImpl.java:52` (`getEmployeeCount`): `countEmployeesByDepartmentId` + `findById` doar pentru `getName()`, două interogări unde ar ajunge una.
- `DepartmentCommandServiceImpl.java:41`: `department.getEmployees().stream()...` pe o entitate încărcată lazy, SELECT separat. Alte `findById` fără graf: `:51`, `:73`, `:85`.
- `EmployeeQueryServiceImpl.java:31` / `:34` (`findAll()` de două ori), `:48`, `:53`, `:61`, `:42` și `EmployeeCommandServiceImpl.java:53` / `:130`: aceeași familie, vezi B1.
- `Department.java:16` `@NamedEntityGraph("Department.withEmployees")`: definit și nefolosit, cod mort.

B3 — Tot fără `@RestControllerAdvice`, a șasea rundă: endpoint-urile noi adaugă o a patra excepție și un 500 nou pe `sort`

`department/services/DepartmentQueryServiceImpl.java:66`

CUM E ACUM.

```
GET /api/department/999/employees       ->  500  (DepartmentNotFound: Department not found)
GET /api/employees/pageable?sort=foo    ->  500  (PropertyReferenceException: No property 'foo' found for type 'Employee')
GET /api/department/999                 ->  500  (NoDepartmentFound)
GET /api/employees (după DELETE /api/department/1..4)  ->  500  (NoEmployeesFound)
GET /api/employees?jobTitle= (tabel gol)               ->  200 []
```

`DepartmentNotFound` e o clasă nefolosită până acum, a patra pentru „departament inexistent” (alături de `NoDepartmentFound`, `DepartmentIdNotFound` și `NoEmployeesFound`). `sort=foo` e o greșeală a clientului, deci ar trebui 400. Iar `GET /api/employees` și `?jobTitle=` pe același tabel gol se contrazic: 500 contra `200 []`.

DE CE. O excepție aruncată din controller sau serviciu urcă până la `DispatcherServlet`. Acolo Spring caută un `@ExceptionHandler` potrivit tipului. Dacă nu găsește, aplică regula de rezervă: orice `RuntimeException` necunoscută e o defecțiune de server, deci 500. Numele clasei (`NotFound`) nu contează, Spring nu-l citește. Contează doar dacă există un handler înregistrat pe acel tip. De-aia fiecare excepție nouă pe care o scrii fără advice e invizibilă pentru client. La `sort=foo` mecanismul e identic: `@PageableDefault` acceptă orice text în `sort`, Spring Data îl validează abia când construiește interogarea, și excepția lui ajunge tot la regula de rezervă.

FIX — o singură clasă, scrisă o dată:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(DepartmentNotFound.class)
    ProblemDetail notFound(RuntimeException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    }
    @ExceptionHandler(PropertyReferenceException.class)
    ProblemDetail badSort(PropertyReferenceException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
    }
}
```

Apoi reduci cele patru excepții de departament la una și o adaugi în același handler. Verificat în scratchpad: `/999/employees` -> 404 `{"detail":"Department not found","status":404,...}`, `sort=foo` -> 400.

+ același bug la (toate reproduse = 500, dacă nu e notat altfel):

- `DepartmentQueryServiceImpl.java:41` `getDepartmentById`: `NoDepartmentFound` -> `GET /api/department/999`.
- `DepartmentQueryServiceImpl.java:52` `getEmployeeCount`: `NoDepartmentFound`, ruta `/employee-count`.
- `DepartmentQueryServiceImpl.java:66`: `DepartmentNotFound` -> `GET /api/department/{id}/employees`; în plus cheamă `getDepartmentById` de două ori.
- `DepartmentCommandServiceImpl.java:51` `updateDepartment` (PUT `/999`), `:73` `changeBudget` (PATCH `/999/budget`), `:85` `deleteDepartment` (DELETE `/999`): `DepartmentIdNotFound`.
- `EmployeeQueryServiceImpl.java:31-32` `getEmployees`: `NoEmployeesFound` pe listă goală; lista goală ar trebui `200 []`.
- `EmployeeQueryServiceImpl.java:42` `getEmployee`: `EmployeeIdNotFound` -> `GET /api/employees/999`.
- `EmployeeQueryServiceImpl.java:60-62` `getAllPageable`: nicio validare a lui `sort`; locul de tratat e `EmployeeController.java:77-81`.
- `EmployeeCommandServiceImpl.java:34` `createEmployee` (POST `/api/employees/999` cu body valid), `:60` `updateEmployee`, `:97` PATCH `/999/salary`, `:110` și `:123` (inclusiv DELETE `/999`).
- Excepțiile fără mapare HTTP (toate extind `RuntimeException`, fără `@ResponseStatus`): `DepartmentIdNotFound`, `DepartmentNotFound`, `NoDepartmentFound`, `EmployeeIdNotFound`, `NoEmployeesFound`.
- Observație secundară, altă regulă: validarea `@Valid` dă 400 fără corp structurat, iar `GET /api/department/count-employees-per-department` dă 400 pentru că ruta e prinsă de `/{id}` (`DepartmentController.java:56` vs `:68`). Un handler pentru `MethodArgumentTypeMismatchException` / `MethodArgumentNotValidException` ar uniformiza răspunsurile.

🟡 Importante

M1 — `EmployeeController.java:70-72`: `@GetMapping(params = "jobTitle")` + `@RequestParam(required = false)` se contrazic. Mapping-ul se potrivește doar când parametrul există, deci ramura `jobTitle == null` din `EmployeeQueryServiceImpl.java:47` e moartă, iar `?jobTitle=` (gol) întoarce toți angajații. Căutarea e și case-sensitive: `?jobTitle=recruiter` -> `[]`. O singură metodă `GET /api/employees` cu `@RequestParam(required = false)` acoperă ambele cazuri.

M2 — `EmployeeController.java:77`: `/pageable` ca segment de path. Paginarea e un parametru al colecției (`GET /api/employees?page=0&size=2`), nu o resursă separată.

M3 — `EmployeeController.java:79`: `sort` acceptă orice câmp al entității, inclusiv `accessCode` și `salary`, care nu apar în răspuns. `?sort=accessCode,desc` -> 200, deci un client poate deduce ordinea codurilor de acces. Default-ul `sort = "salary"` expune la fel ierarhia salariilor. Fă o listă albă de câmpuri sortabile.

M4 — `DepartmentQueryServiceImpl.java:66`: `== null` pe o metodă derivată care întoarce entitatea. `Optional` + `orElseThrow` e tiparul pe care l-ai generalizat singur în runda 4 (B6). Se rezolvă odată cu B2.

M5 — excepții: `DepartmentNotFound`, `NoDepartmentFound` și `DepartmentIdNotFound` sunt toate folosite acum, pe trei rute diferite, pentru același caz (R5 M4 agravat). Păstrează una.

M6 — adnotări de web în interfețe de serviciu, R5 M1 neatins: `DepartmentCommandService.java:4-5` (`PathVariable` , `RequestBody`) și `DepartmentQueryService.java:3` (`RequestParam`).

M7 — cod mort: `getEmployeeCount` (`DepartmentQueryServiceImpl.java:50`) nu mai e chemat după fix-ul R5 B1 · `ListDepartmentEmployeeCount` · `changeSalaryThruUrl` (`EmployeeCommandServiceImpl.java:109` + interfața) · importuri nefolosite în `EmployeeController.java:14-15` (`javax.print.attribute.standard.PageRanges` e nou, adus de autocomplete) · `import Page` nefolosit în `EmployeeQueryService.java` .

M8 — `DepartmentEmployeesResponse` repetă `departmentName` în fiecare angajat, sub un părinte care are deja `departmentName` (R5 M6, reapărut în DTO-ul nou).

M9 — deschise din rundele anterioare, reproduse azi: `PUT /api/employees/1` cu emailul altui angajat -> 200, emailul rămâne neschimbat și nu primești nicio eroare (R4 B4) · `DELETE /api/department/{id}` șterge angajații (R4 B7; după ștergerea celor 4 departamente, tabelul `employee` e gol) · POST răspunde 200 în loc de 201 · `/api/department` singular · `mvnw` fără bit de execuție (`-rw-r--r--`) · nume de departament neunic.

🟢 Cleanups

- **C1** — `pagebale` (`EmployeeController.java:79`) -> `pageable` .
- **C2** — `PageResponse` e generic, dar stă în `employee/dtos` ; mută-l într-un pachet comun.
- **C3** — `new PageResponse<T>` (spațiu dublu), rânduri goale în plus la finalul `DepartmentQueryServiceImpl` și `DepartmentController` .
- **C4** — `getDepartmentById` e numele a două lucruri diferite: metoda din repository (cu graf, entitate) și cea din serviciu (`findById` , DTO). Dispare odată cu B2.

✅ **Ce e bine** - R5 B1 închis curat: `countEmployeesPerDepartment` cu `group by` emite 1 SQL, iar `Legal` apare cu `employeeCount: 0` . - Ai ales `@EntityGraph(attributePaths = …)` , nu forma cu nume, adică varianta pe care o verifică pornirea aplicației. - `PageResponse.from(page, mapper)` nu expune `PageImpl` în JSON, iar `findAll(pageable)` dă `totalElements: 7` corect (count-ul nu e umflat de join). - `GET /api/department/{departmentId}/employees` e o formă REST naturală, în direcția M2 din runda 5.

Before / After pentru 🔴

	Acum	Cum ar trebui
B1 `EmployeeRepository`	`Page<Employee> findAll(Pageable p)` moștenit, fără graf: 5 SQL pe `size=7`	`@EntityGraph(attributePaths = "department") Page<Employee> findAll(Pageable p);` redeclarat, plus același graf pe `findByJobTitle` și `findById` : 2 SQL
B1 `getEmployees()`	`findAll().isEmpty()` apoi `findAll().stream()` : două selecturi	`List<Employee> l = employeeRepository.findAll(); if (l.isEmpty()) ...` : un singur apel
B2 `DepartmentRepository`	`Department getDepartmentById(Long id)` cu graf, plus `@NamedEntityGraph` nefolosit	`@EntityGraph(attributePaths = "employees") Optional<Department> findById(Long id);` , fără `@NamedEntityGraph`
B2 `DepartmentQueryServiceImpl:66-68`	`if (repo.getDepartmentById(id) == null) throw ...; Department d = repo.getDepartmentById(id);`	`Department d = repo.findById(id).orElseThrow(DepartmentNotFound::new);`
B3 erori	4 excepții de departament, zero handlere: `/999/employees` = 500, `sort=foo` = 500	un `@RestControllerAdvice` : `DepartmentNotFound` -> 404, `PropertyReferenceException` -> 400; o singură excepție de departament

---

Ordinea de atac

1. **B3** — advice-ul global. E cea mai veche constatare deschisă, din runda 1, și face următoarele verificabile.
2. **B1** — `@EntityGraph(attributePaths = "department")` pe `findAll(Pageable)` , `findByJobTitle` , `findById` ; apoi numără liniile `Hibernate:` pe fiecare cerere.
3. **B2** — `findById` cu graf în loc de `getDepartmentById` , un singur apel în serviciu, șterge `@NamedEntityGraph` .
4. R5 B4 — `DepartmentEmployeeCount` în loc de `Map` .
5. M1–M3 pe `EmployeeController` , apoi restanțele (email duplicat -> 409, cascade la delete).

Q&A — runda 6

1. `GET /api/employees/pageable?size=7` emite 3 SELECT-uri pe `department` , nu 7, deși pagina are 7 angajați. Ce anume din Hibernate face ca al doilea angajat din IT să nu mai declanșeze o interogare, și ce număr din datele tale crește când endpoint-ul devine lent în producție?
2. În `getAllEmployeesByDepartmentId` chemi `getDepartmentById` de două ori și vezi în consolă două SELECT-uri identice. De ce cache-ul de prim nivel nu-l scutește pe al doilea, în timp ce două `findById` (`em.find`) pe același id, în aceeași tranzacție, ar ajunge o singură dată la bază?
3. `DepartmentNotFound` și `PropertyReferenceException` ies amândouă ca 500. Ce caută Spring când o excepție ajunge la `DispatcherServlet` , și de ce numele clasei tale (`...NotFound`) nu influențează deloc codul HTTP?

Următorul pas: repară în ordinea de mai sus și scrie-mi „next".

Runde anterioare

Runda 5 — 2026-09-03 — sha `e642a5f`

- B1 🔴 N+1 scris de mână în `getAllEmployeeCounts` (6 SQL) -> ✅ **ÎNCHIS** (`group by` , 1 SQL)
- B2 🔴 `@NamedEntityGraph` declarat și nefolosit; `GET /api/department/{id}` și `/api/employees/{id}` cu interogare în plus -> 🔴 **DESCHIS** (vezi B2 runda 6)
- B3 🔴 fără `@RestControllerAdvice` -> 🔴 **DESCHIS, a șasea rundă** (vezi B3 runda 6)
- B4 🔴 `Map<String,Object>` cu cheia `"emmployee count"` (`DepartmentController.java:57-62`) -> 🔴 **DESCHIS**
- M1 web în servicii -> deschis · M2 nume inversate la endpoint-urile de count -> deschis · M3 `getEmployeeCount` cu 2 interogări -> acum cod mort · M4 excepții -> agravat (`DepartmentNotFound` intră în uz) · M5 cod mort -> deschis, plus import nou `PageRanges` · M6 `departmentName` repetat -> reapărut în `DepartmentEmployeesResponse`
- Închise din R4 în runda 5: B1 `@NotBlank` pe `Double budget` · B2 `PATCH .../salary` returna salariul vechi · B3 salariu negativ prin path variable · B6 `orElseThrow()` gol (toate 7 locurile, după `grep`) · M3 `existsById` + `findById`

Runda 4 — 2026-09-03 — sha `24e6f9c`

- B1 🔴 `@NotBlank` pe `Double budget` omora `PUT /api/department/{id}` -> ✅ **ÎNCHIS** (`@NotNull`)
- B2 🔴 `PATCH .../salary` returna salariul vechi -> ✅ **ÎNCHIS**
- B3 🔴 `PATCH /{id}/{newSalary}` accepta salariu negativ -> ✅ **ÎNCHIS** (mapping șters)
- B4 🔴 `PUT` cu email duplicat răspunde 200 tăcut, în loc de 409 -> 🔴 **DESCHIS**
- B5 🔴 fără `@RestControllerAdvice` -> 🔴 **DESCHIS**
- B6 🔴 `orElseThrow()` gol în 4 din 7 locuri -> ✅ **ÎNCHIS, toate 7**
- B7 🔴 `DELETE` pe departament șterge angajații -> 🔴 **DESCHIS**
- M1 `POST /api/employees/{departmentid}` -> deschis · M2 `changeSalary` vs `changeSalaryThruUrl` -> ✅ parțial · M3 gărzi moarte în `updateEmployee` -> deschis · M4 `EmployeeResponse.from` presupune departament -> deschis · M5 POST 200 în loc de 201 -> deschis · M6 `updatedRows` expus -> deschis · M7 `@Transactional` jakarta fără `readOnly` -> deschis · M8 adnotări de web în servicii -> agravat · M9 `/api/department` singular -> deschis · M10 nume departament neunic + `mvnw` fără bit exec -> deschise
- C1–C8 -> majoritatea deschise; C3 (typo „pozitive" în `EmployeeUpdateRequest.java:23`) -> deschis

Runda 3 — 2026-09-02 — sha `edebe10`

- B1 🔴 `changeBudget` făcea `update ... where d.name` -> ✅ **ÎNCHIS** (`updateBudgetById`)
- B2 🔴 fără `@RestControllerAdvice` , 4 excepții toate 500 -> 🔴 **DESCHIS**
- B3 🔴 `orElseThrow()` gol -> ✅ 3 din 7 închise, restul în R4 B6
- B4 🔴 `DELETE` pe departament șterge și angajații -> 🔴 **DESCHIS**
- B5 🔴 nu se putea crea un angajat prin API -> ✅ **ÎNCHIS**
- B6 🔴 `isEmpty()+throw` și `findAll()` dublat la angajați -> 🔴 **DESCHIS**
- M1 web în interfața de serviciu -> deschis · M2 `PUT` cu semantică de `PATCH` -> ✅ închis · M3 `existsById` + `findById` -> deschis · M4 trei excepții pentru aceeași idee -> deschis · M5 `@Transactional` jakarta -> deschis · M6 POST 200 -> deschis · M7 postman pe `/api/company` -> ✅ închis · M8 `/api/department` singular -> deschis · M9 `updatedRow` expus -> deschis · M10 nume departament neunic -> deschis · M11 `mvnw` fără bit exec -> deschis
- ⭐ `@Modifying(clearAutomatically = true, flushAutomatically = true)` pus singur, fără să i se ceară.

Runda 2 — 2026-09-02 — sha `e8f05ad`

- B1 🔴 Nu compila: `interface EmployeeComandService` cu fișier redenumit; cele 19 `cannot find symbol: getId()` erau colaterale Lombok -> ✅ **ÎNCHIS**
- B2 🔴 Commit-ul `e8f05ad` conținea doar rename-ul -> ✅ munca reală a venit în `2e16475` + `edebe10`

- B3–B7 🔴 restanțele din runda 1 -> 3 închise în runda 3 (`@Valid` , listă goală, `jobTitle`), 2 deschise (advice, endpoint employee)
- M1 🟡 cele 4 DTO-uri goale -> ✅ **ÎNCHIS** (toate `record` , cu validare)

Runda 1 — 2026-09-02 — sha `943d7ad`

- B1 🔴 `@Valid` lipsă pe `List<EmployeeCreateRequest>` -> ✅ închis (lista eliminată) · B2 🔴 zero `@RestControllerAdvice` -> 🔴 **DESCHIS** · B3 🔴 listă goală = eroare -> ✅ la departamente, reapărut la angajați · B4 🔴 fără `EmployeeController` -> parțial · B5 🔴 `jobTitle` 50 vs 20 -> ✅ închis
- M1 postman rupt -> ✅ · M2 dialect hardcodat -> ✅ · M3 dublă interogare -> ✅ la departamente, reapărut la angajați · M4 `mvnw` fără bit exec -> deschis · M5 POST 200 -> deschis · M6 fără `@Transactional` -> ✅ · M7 `orElse(null)` + `if/else` -> deschis · M8 nume departament neunic -> deschis · M9 naming -> ✅ parțial · M10 `/api/department` singular -> deschis

✅ **Ce e bine — nu strica astea la refactor:** `equals` / `hashCode` pe entități scrise corect pentru JPA · constructor `protected` + constructor de business, fără `@Setter` pe `id` · helper-ele bidirecționale `addEmployee` / `removeEmployee` · `left join fetch` dă un singur SELECT · separarea Command/Query · `@Modifying(clearAutomatically, flushAutomatically)` .

Regulă de reținut — `join fetch` sau `@EntityGraph` ?

> **Scrii tu JPQL-ul -> `join fetch` . Nu-l scrii tu, sau returnezi `Page` -> `@EntityGraph(attributePaths = …)` .**

- La paginare, `join fetch` pe `Page` face ca interogarea de `count` să numere rânduri de join: 4 departamente raportate ca `totalElements: 8` , `totalPages: 4` . `@EntityGraph` dă count curat (4 / 2).
- `join fetch` rămâne alegerea când filtrezi colecția încărcată (`left join fetch d.employees e where e.salary > :prag`); în `getDepartmentsWithEmployees()` nu-l schimba.
- `@EntityGraph(attributePaths = "typooo")` crapă zgomotos la pornire; `@EntityGraph(value = "NumeInexistent")` e ignorat în tăcere. Folosește `attributePaths` , nu grafuri numite.